Node.js Portfolio Deployment using Jenkins CI/CD, GitHub & AWS EC2

Project Overview

This project demonstrates the automated deployment of a Node.js portfolio application using **Jenkins, GitHub, and AWS EC2**. A Jenkins pipeline defined in a `Jenkinsfile` automates code checkout, dependency installation, application building, testing, and deployment to an EC2 instance.

The goal is to simplify application deployment, reduce manual effort, and implement a continuous integration and continuous delivery (CI/CD) workflow.

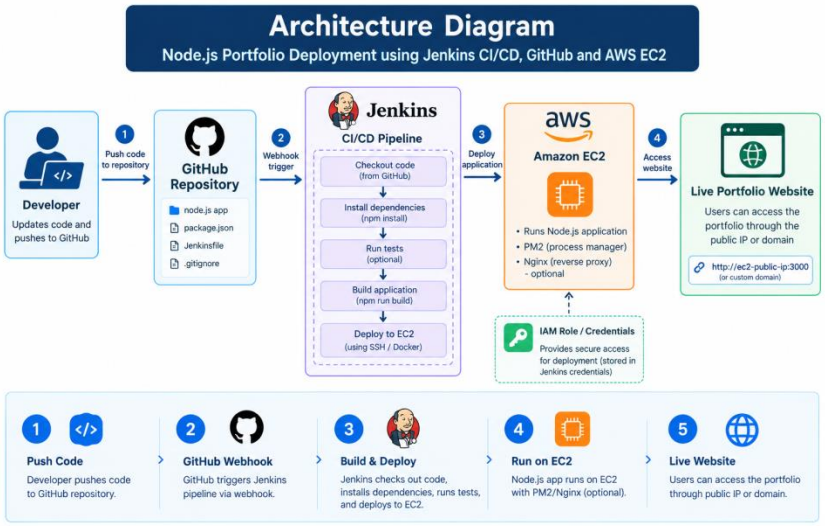
Project Objectives

- Automate Node.js application deployment using Jenkins.
- Integrate GitHub with Jenkins using webhooks.
- Define pipeline stages using a Jenkinsfile.
- Host the Node.js application on AWS EC2.
- Use Git and GitHub for source code management.
- Apply security best practices for deployment credentials.

Technologies Used

Technology	Purpose
Node.js	Application runtime
Git	Version control
GitHub	Source code repository
Jenkins	CI/CD automation
Jenkinsfile	Pipeline definition
GitHub Webhooks	Trigger Jenkins builds
AWS EC2	Application hosting
npm	Dependency management

Architecture Diagram



Architecture Workflow

1. The developer modifies the Node.js portfolio application.
2. Code is pushed to the GitHub repository.
3. A GitHub webhook triggers the Jenkins pipeline.
4. Jenkins checks out the latest source code.
5. Jenkins installs dependencies and runs available tests.
6. The pipeline builds the application if a build script is configured.
7. Jenkins deploys the application to AWS EC2 using SSH or Docker.
8. Users access the running portfolio through the EC2 public IP or configured domain.

CI/CD Pipeline Workflow

Stage 1: Source Code Management

The developer pushes updated code to GitHub.

Stage 2: GitHub Webhook Integration

Configure a GitHub webhook to notify Jenkins when code is pushed.

In GitHub, navigate to:

Repository → Settings → Webhooks → Add webhook

Set the payload URL to your Jenkins webhook endpoint:

Stage 3: Build and Test

Jenkins checks out the source code and installs the required npm dependencies.

Stage 4: Deployment to AWS EC2

Jenkins transfers the application to the EC2 instance using SSH or deploys a Docker image, depending on the selected deployment method.

Stage 5: Application Access

Once deployment is successful, users can access the portfolio using the configured public IP address or domain.

AWS EC2 Configuration

1. Launch an EC2 instance with a suitable Linux operating system.
2. Configure SSH access for the Jenkins deployment agent.
3. Configure the EC2 security group to allow only the required inbound traffic.
4. Deploy the application and ensure it runs persistently using an appropriate process manager, such as PM2, or a container restart policy.
5. Optionally configure Nginx and a domain name with HTTPS.

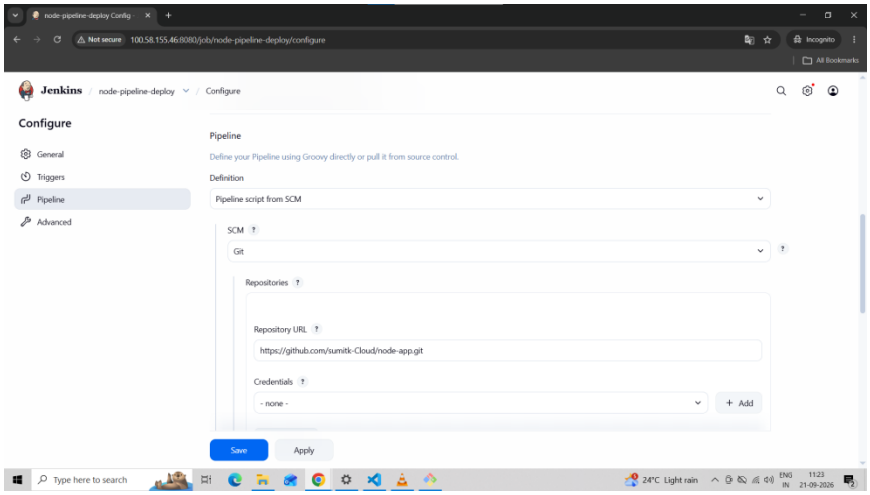
Security Best Practices

- Use Jenkins Credentials to store sensitive deployment credentials.
- Restrict SSH access to trusted sources.
- Apply least-privilege permissions to AWS IAM users and roles.
- Avoid committing `.env` files, passwords, private keys, or other secrets to GitHub.
- Keep Jenkins, Node.js, dependencies, and the operating system updated.
- Use HTTPS for public-facing applications.
- Avoid exposing the Jenkins dashboard directly to the public internet without appropriate access controls.

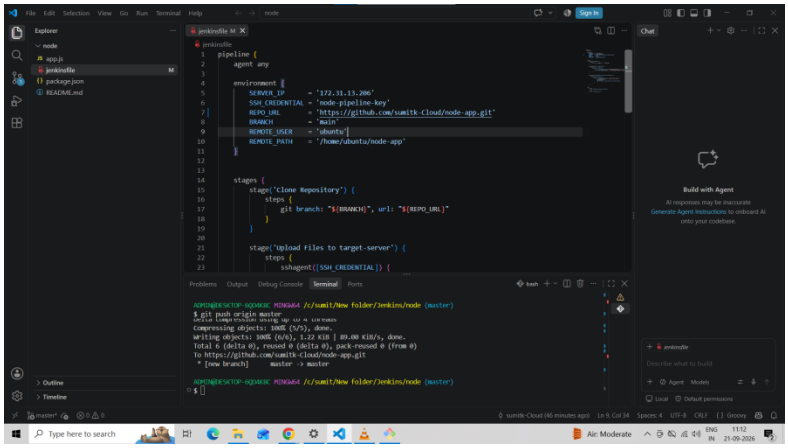
Project Screenshots

.

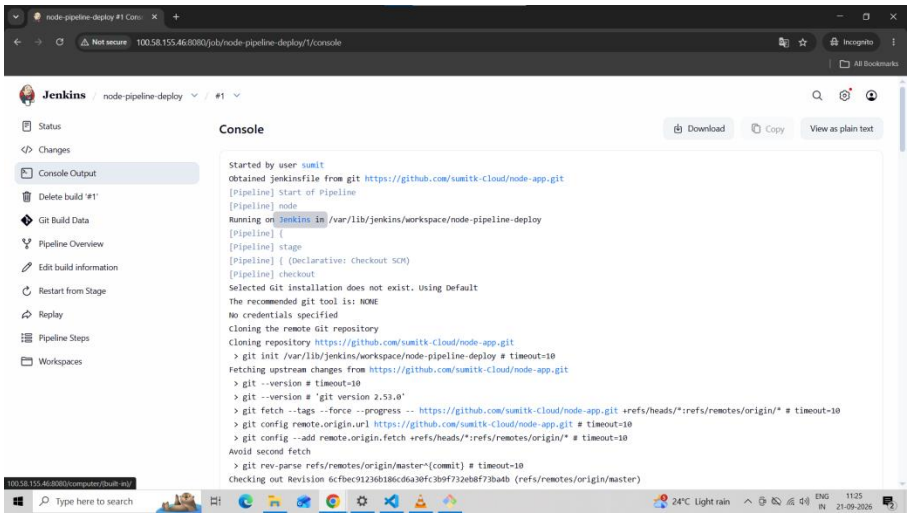
1. GitHub Repository



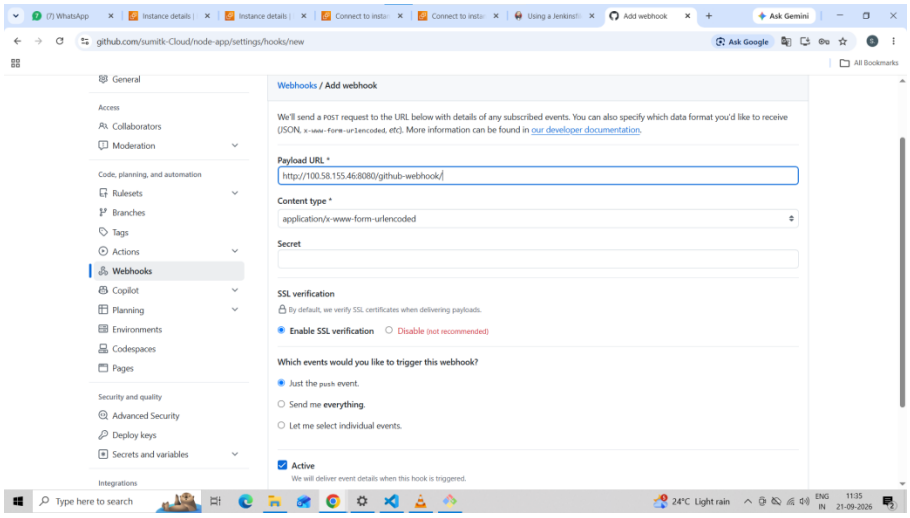
2. Jenkins Pipeline



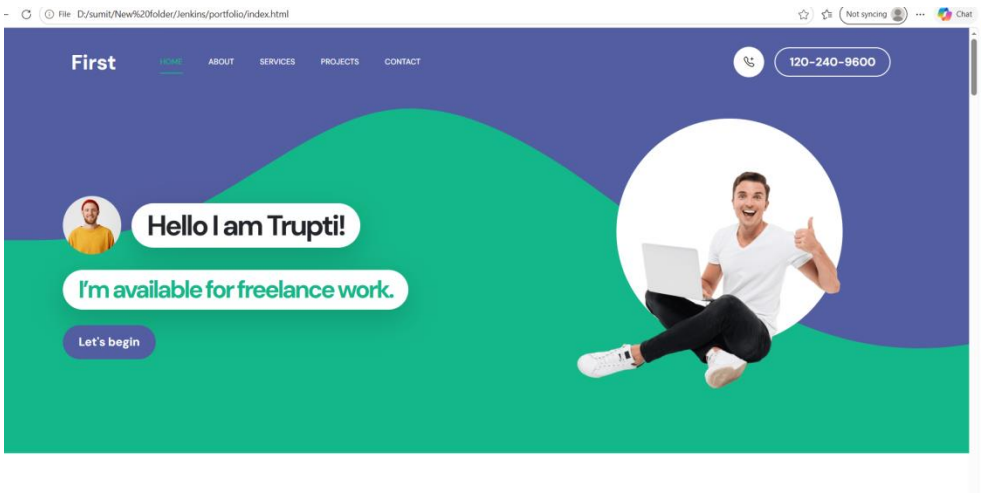
3. Successful Build



Webhook add



4. Live Portfolio Website



Key Learnings

- Implementing CI/CD with Jenkins.
- Integrating GitHub webhooks with Jenkins.
- Managing Node.js dependencies and build stages.
- Deploying applications to AWS EC2.
- Managing deployment credentials securely.
- Understanding automated application delivery.

Future Enhancements

- Containerize the application using Docker.

- Add Docker image publishing to Amazon ECR.
- Deploy using Amazon ECS or Kubernetes.
- Configure Nginx, a custom domain, and HTTPS.
- Add automated code quality and security scans.
- Introduce monitoring and notifications for failed deployments.

Author

Sumit Kamble

- GitHub: sumitk-Cloud
- LinkedIn: Sumit Kamble
- Email: sumitkamble5550@gmail.com
